Agenda

- Exception Handling
  - Exceptions
  - Errors
  - Exception Chaning
  - Custom Exceptions
- ~~clone()~~
- ~~Date/LocalDate/Calender~~

Exception Handling

- Exceptions represents runtime problems
- Java have divided the exceptions into two categories
  1. Error
  2. Exception
- java.lang.Throwable is the super class of all the Errors and Exceptions

```
- Throwable (Super class of all Errors and Exceptions)
    - Error
        - VirtualMachineError
            - OutOfMemoryError
            - StackOverflowError
        - IOError
    - Exception (Checked Exception - Checked at compile time,forced to handle)
        - CloneNotSupportedException
        - IOException
        - InterruptedException
        - RuntimeException (Unchecked Exception - Not Checked By Compiler)
            - ArithmeticException
            - ClassCastException
            - IndexOutOfBoundsException
                - ArrayIndexOutOfBoundsException
                - StringIndexOutOfBoundsException
            - NegativeArraySizeException
            - NoSuchElementException
                - InputMismatchException
            - NullPointerException
```


Errors

- Errors are generated due to runtime environment.
- It can be due to problems in RAM/JVM for memory management or like crashing of harddisk, etc.
- We cannot recover from such errors in our program and hence such errors should not be handled.
- we can write a try catch block to handle such errors but it is recommended not to handle such errors.

```
public static void m1() {
        double arr[] = new double[999999999]; // OutOfMemoryError
        System.out.println("Program Executed Successfully");
}

static void m2() {
        int num1 = 10;
        double num2 = 20;
        long num3 = 30;
        m2(); // StackOverFlowError
}

public static void main(String[] args) {
        //m1();
    m2();
}
```

Exception

- Exceptions represents logical problems in code.
- If not handled in the current method it is sent back to the calling method.
- To perform exception handling java have provided below keywords
    - try
    - catch
    - throw
    - throws
    - finally
- Java operators/APIs throw pre-defined exception if runtime problem occurs.
- Exceptions need to handled otherwise the it terminates the program by throwing that exception.
- we use try catch block to handle the exception. Inside try block keep all the method calls that generate exception and handle the exception inside catch block

```
public static void divide(int numerator, int denominator) {
  int result = numerator / denominator;
  System.out.println("Division - " + result);
}

public static void main(String[] args) {
  try {
    divide(10, 0);
  } catch (ArithmeticException exception) {
    System.out.println("Denominator cannot be 0");
  }
  System.out.println("Program completed succesfullly...");
}
```

- When exception is raised, it will be caught by nearest matching catch block.

- we can provide multiple catch block for a single try block
- If no matching catch block is found, the exception will be caught by JVM and it will abort the program.

```
public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);
  try {
    System.out.print("Enter numerator - ");
    int numerator = sc.nextInt();

    System.out.print("Enter denominator - ");
    int denominator = sc.nextInt();

    int result = numerator / denominator;
    System.out.println("Division - " + result);
  } catch (ArithmeticException exception) {
    System.out.println("Denominator cannot be 0");
  } catch (InputMismatchException e) {
    System.out.println("Wrong input entered");
  }
  System.out.println("Program completed succesfullly...");
}
```

- We can handle multiple exceptions in a single catch block using either

1. multi-catch block

```
public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);
  try {
    System.out.print("Enter numerator - ");
    int numerator = sc.nextInt();
    System.out.print("Enter denominator - ");
    int denominator = sc.nextInt();
    int result = numerator / denominator;
    System.out.println("Division - " + result);
  } catch (ArithmeticException | InputMismatchException ex) {
    ex.printStackTrace();
  }
  System.out.println("Program completed succesfullly...");
}
```

2. Generic catch block

```
public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);
  try {
    System.out.print("Enter numerator - ");
    int numerator = sc.nextInt();
```

```
      System.out.print("Enter denominator - ");
      int denominator = sc.nextInt();
      int result = numerator / denominator;
      System.out.println("Division - " + result);
    } catch (Exception ex) { // concept of upcasting used
      ex.printStackTrace();
    }
    System.out.println("Program completed succesfullly...");
  }
```

- when exceptions are generated if we do not handle it then program terminates,however the resources that are in used should be closed.
- the resources can be closed in finally block that can be used with a try block.

```
public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);
  try {
    System.out.print("Enter numerator - ");
    int numerator = sc.nextInt();
    System.out.print("Enter denominator - ");
    int denominator = sc.nextInt();
    int result = numerator / denominator;
    System.out.println("Division - " + result);
  }finally {
    sc.close();
    System.out.println("Resource Closed..");
  }
  System.out.println("Program completed succesfullly...");
}
```

- if the classes have implemented AutoCloseable interface we can also use try with resource with the try block.

```
public static void main(String[] args) {
  try (Scanner sc = new Scanner(System.in)){
    System.out.print("Enter numerator - ");
    int numerator = sc.nextInt();

    System.out.print("Enter denominator - ");
    int denominator = sc.nextInt();

    int result = numerator / denominator;
    System.out.println("Division - " + result);
}
    System.out.println("Program completed succesfullly...");
}
```

- when we use try block then,it should atleast have
    1. a catch block
    2. a finally block
    3. try with resource

Exception Handling Keywords

- 1. try
    - Code where runtime problems may arise should be written in try block.
    - try block must have one of the following
        - catch block
        - finally block
        - try-with-resource
    - it Can be nested in try, catch, or finally block
- 2. catch
    - Code to handle error/exception should be written in catch block.
    - Argument of catch block must be Throwable or its sub-class.
    - Generic catch block -- Performs upcasting -- Should be last (if multiple catch blocks)
- 3. finally
    - Resources are closed in finally block.
    - Executed irrespective of exception occurred or not.
    - finally block not executed if JVM exits (System.exit()).
- 4. "throw" statement
    - Throws an exception/error i.e. any object that is inherited from Throwable class.
    - Can throw only one exception at time.
    - All next statements are skipped and control jumps to matching catch block.
- 5. throws
    - Written after method declaration to specify list of exception not handled by called method and to be handled by calling method.
    - Writing unhandled checked exceptions in throws clause is compulsory.
    - Sub-class overridden method can throw same or subset of exception from super-class method.

Exception Types

- There are two types of exceptions

1. Checked exception -- Checked by compiler and forced to handle
2. Unchecked exception -- Not checked by compiler

- Exception class and all its sub classes except Runtime exception class are all Checked Exception
- Runtime Exception and all its sub classes all unchecked exceptions

1. Checked exception

- Exception and all its sub classes (except RuntimeException) are checked exceptions.
- Typically represents problems arised out of JVM/Java i.e. at OS/System level.
- IOException, SQLException, etc..
- Compiler checks if the exception is handled in one of following ways.

- Matching catch block to handle the exception.

```
void someMethod() {
  try {
  // file io code
  }
  catch(IOException ex) {
  }
}
```

- throws clause indicating exception to be handled by calling method.

```
void someMethod() throws IOException {
  // file io code
}

void callingMethod() {
  try {
          someMethod();
  }
  catch(IOException ex) {
  }
}
```

```
public static void method1(){
  for (int i = 0; i < 10; i++) {
    try {
      Thread.sleep(1000); // Checked exceptions are compulsary to handle
    } catch (InterruptedException e) {
      e.printStackTrace();
    }
    System.out.println("i = " + i);
  }
}

public static void method2() throws InterruptedException
{
  for (int i = 0; i < 10; i++) {
      Thread.sleep(1000);
      System.out.println("i = " + i);
  }
}

public static void main(String[] args) // throws InterruptedException //throwing
exception from main in bad practice
{
  method1()
  // method2(); // all exceptions finally should be handled in main
```

```
  // correct way
  try {
     method2();
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
  System.out.println("Program completed succesfullly...");
}
```

2. Unchecked exception

- RuntimeException and all its sub classes are unchecked exceptions.
- Typically represents programmer's or user's mistake.
- NullPointerException, NumberFormatException, ClassCastException, etc.
- Compiler doesn't provide any checks -- if exception is handled or not.
- Programmer may or may not handle (catch block) the exception. If exception is not handled, it will be caught by JVM and abort the application.

Generating Exception (throw keyword)

- We can generate the exception as per the requirement.
- It can be checked or unchecked exception
- To generate an exception use throw keyword followd by the object of the required Exception class

```
class Date {
private int day;
private int month;

public void setDay(int day) {
  if (day <= 0 || day > 31)
    throw new RuntimeException(); // generate an unchecked exception
  this.day = day;
}

public void setMonth(int month) {
  if (month <= 0 || month > 12)
    throw new RuntimeException("Month should be between 1 to 12"); // generate an
unchecked exception with message
  this.month = month;
}
}
```

```
class Date {
private int day;
private int month;

public void setDay(int day) throws Exception {
  if (day <= 0 || day > 31)
```

```
    throw new Exception(); // generate an checked exception
  this.day = day;
}

public void setMonth(int month) throws Exception {
  if (month <= 0 || month > 12)
    throw new Exception("Month should be between 1 to 12"); // generate an checked
exception with message
  this.month = month;
}
}
```

Exception chaining

- Sometimes an exception is generated due to another exception.
- For example, database SQLException may be caused due to network problem SocketException.
- To represent this an exception can be chained/nested into another exception.
- If method's throws clause doesn't allow throwing exception of certain type, it can be nested into another (allowed) type and thrown.

```
public static void getEmployees() throws SQLException {
        // logic to get all the employees from database
        boolean connection = false;
        if (connection) {
            // fetch data
            boolean data = false;
            if (data)
                System.out.println(data);
            else
                throw new SQLException("Data not found");
        } else
            throw new SQLException("failed", new SocketException("Connection
rejected"));
    }
```

User defined exception class

- If pre-defined exception class are not suitable to represent application specific problem, then user-defined exception class should be created.
- User defined exception class may contain fields to store additional information about problem and methods to operate on them.
- Typically exception class's constructor call super class constructor to set fields like message and cause.
- If class is inherited from RuntimeException, it is used as unchecked exception. If it is inherited from Exception, it is used as checked exception.

```
// custom checked exception
class InvalidTimeException extends Exception {
```

```
    public InvalidTimeException() {
    }

    public InvalidTimeException(String message) {
        super(message);
    }
}

// custom unchecked exception
class InvalidDateException extends RuntimeException {

    public InvalidDateException() {
    }

    public InvalidDateException(String message) {
        super(message); // ctor chaining
    }
}
```